# Forensics in Digital Security (FDS)

# Digital Investigator

# Data Representation & Number Systems

# Objectives

**Define a bit and describe how a series of bits represents data**

**Know the terms bits, bytes, and word**

**Understand how data is represented in computers.**

# Objectives

**Describe the various encoding schemes used to represent digital data**

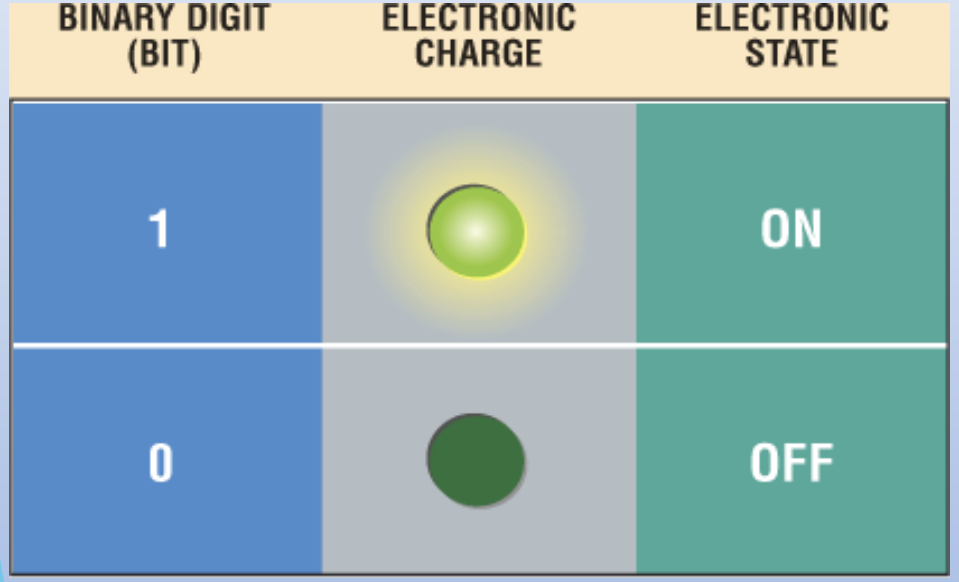
**Understand how the number systems works in a computer**

**Describe the common number systems and how to convert from one number system to another**

# Data Representation

Recall: How do computers represent data?

- **Most computers are digital**




- Recognize only two discrete states: on or off
- Use a **binary system** to recognize two states
- Use Number system with two unique digits: 0 and 1, called **bits** (short for binary digits)

# Data Representation

Eight bits are grouped together to make one **byte.**

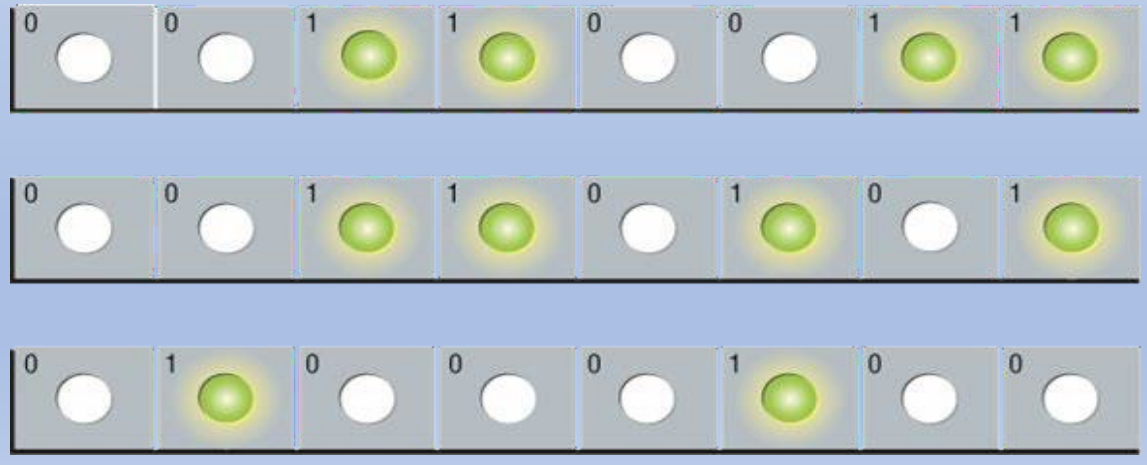
**A table displaying the values of computer memory**

| | |
|---|---|
| Bit | 1 or 0 |
| Byte | 8 Bits |
| Kilobyte (Kb) | 1024 bytes |
| Megabyte (Mb) | 1024 kilobytes |
| Gigabyte (Gb) | 1024 megabytes |
| Terabyte (Tb) | 1024 gigabytes |

# Data Representation

## What is a byte?

- **Eight bits grouped together as a unit**
- **Provides enough different combinations of 0s and 1s to represent 256 individual characters**
  - Numbers
  - Uppercase and lowercase letters
  - Punctuation marks



# Common Coding Systems

Character Representation

**EBCDIC**
8 bits

**ASCII**

**UNICODE**
Max 16 bits

7 bits
- **represent non graphical data**
- **odd and even parity for error checking**

8 bits
- **represent graphical data**

HTTP://WWW.ASCIITABLE.COM/

# Common Coding Systems

- Extended Binary Coded Decimal Interchange Code (EBCDIC) is 8 bits and developed by IBM®
- 7 bits American Standard Code for Information Interchange (ASCII) is used to represent non graphical data
- 8 bits ASCII is used to represent graphical data
- Unicode uses a maximum of 16 bits to represent a character

- An 8 bit ASCII coding system can represent (2 to the power of 8 ie $2^8 = 256$) ***256*** symbols/characters

# Data Representation

the capital letter **D** (shift+D key) on the keyboard.

## How is a letter converted to binary form and back?

**Step 1.**
The user presses

**Step 2.**
An electronic signal for the capital letter **D** is sent to the system unit.

**Step 3.**
The signal for the capital letter **D** is converted to its ASCII binary code ($01000_9100$) and is stored in memory for processing.

**Step 4.**
After processing, the binary code for the capital letter **D** is converted to an image, and displayed on the output device.

# What's the diff between 7 and 8 bits ASCII?

- As most PCs store data in 8 bits, an additional bit is appended to the 7 bits ASCII code.
  - For example, the character A is $1000001_2$.
    In 7 bits ASCII, it will add an additional 0 to make it a full byte $01000001_2$ *(remember 8 bits make 1 byte)*

## Scenario: Exam Grade Entry

- You press the B key on your computer's keyboard to record a B grade for a student.
  ASCII encoder is used to transmit the 7-bit code of B which is **1000010** to the CPU.
  If electrical interference on the line causes the fifth bit to change from 0 to 1, the CPU will receive the message 1000110 which is the ASCII code for F.

- Oh oh…the CPU will think that you have typed in the letter F instead!
  Poor student got a F instead of a B. So how to prevent this?

# Parity Bit – for error checking

- Solution: Use **parity bit**

- An additional bit is automatically set to either 0 or 1 to force the number of 1-bits in each byte to be either an even or an odd number.

# Odd Parity

- When odd parity is used, the number of 1s in a 8 bits code is odd

For example, A is represented by $01000001_2$

*(The additional underlined bit is the parity bit)*

Let's go back to the scenario.

After adding the odd parity bit, the letter B should be 11000010 and sent to the CPU.

# Odd Parity

If electrical interference occurs and the same bit has been flipped as previously, the CPU will receive the bit pattern as 11000110, which has EVEN number of ‘1’s.

(red bit is the erroneous bit)

The CPU expects an odd number of ‘1’s and hence it knows that an error has occurred during transmission!

# Odd Parity

Exercise – Fill in the parity bit for $1100111_2$

Ans: <u>0</u>1100111

*(Note: The underlined bit is the parity bit)*

# Even Parity

- When <u>even</u> parity is used, the <u>number of 1s</u> in a the 8 bits code is <u>even</u>

  For example, A is represented by <u>0</u>$1000001_2$

- Exercise – Fill in the parity bit for $1110110_2$

  Ans: <u>1</u>1110110

  *(Note: The underlined bit is the parity bit)*

- Suppose you receive a binary bit word “0101” and you know you are using an odd parity.

- Is there error in this word?

- The answer is yes:
  - There are 2 1-bit, which is an even number
  - We are using an odd parity
  - So there must have an error.

## Exercise

- Suppose you are using an **ODD** parity.
- What should the binary word “1010” look like after you add the parity bit?

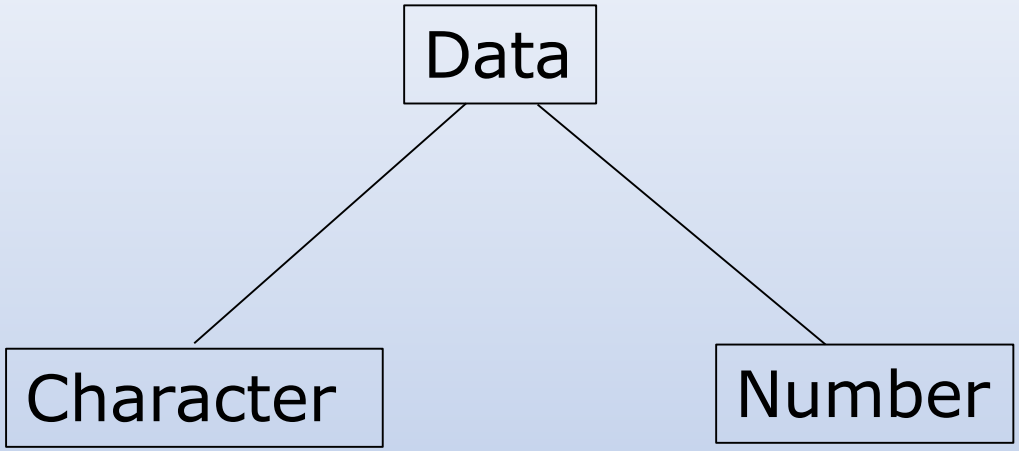
- Answer:
  - There is an even number of 1-bits.
  - So we need to add another 1-bit
  - Our new word will look like “1010**(1)**”.

# Number Systems

# Learning Objectives

- Understand how the number systems works in a computer.
- Understand the arithmetic and logical operations in computers.

# Types of Data Representation




- We have covered how computer represent character previously, how about numbers?

- **Binary**, **decimal** and **hexadecimal** are some of the common number systems used in computer systems

# Number Systems

- **Binary**
  Base: 2 Digits: 0,1

- **Decimal**
  Base: 10 Digits: 0,1,2,3,4,5,6,7,8,9

- **Hexadecimal (Hex)**
  Base: 16 Digits: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

# Counting numbers

Decimal, binary and hexadecimal representations of the numbers 1 to 24 are shown below.

# Base-10 (decimal) arithmetic

- Uses the ***ten*** numbers from 0 to 9
- Each column represents a power of ***10***

**Thousands ($10^3$) column**

**Hundreds ($10^2$) column**

**Tens ($10^1$) column**

**Ones ($10^0$) column**

$1999_{10}$

$= 1 \times 10^3 + 9 \times 10^2 + 9 \times 10^1 + 9 \times 10^0$

$= 1000 + 900 + 90 + 9$

$= 1999$

# Base-2 (binary) arithmetic

- Uses the ***two*** numbers from 0 to 1
- Every column represents a power of ***2***

**Eights ($2^3$) column**

**Fours ($2^2$) column**

**Twos ($2^1$) column**

**Ones ($2^0$) column**

$1001_2$

$= 1x2^3 + 0x2^2 + 0x2^1 + 1x2^0$

$= 8 + 0 + 0 + 1$

$= 9$

# Converting from base-10 to base-2 (Method 1)

| $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ | | |
|---|---|---|---|---|---|---|---|---|
| *64* | *32* | *16* | *8* | *4* | *2* | *1* | | |
| | | 1 | 0 | 0 | 0 | 0 | = | 16 |
| | 1 | 1 | 0 | 1 | 1 | 1 | = | 55 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 | = | 75 |
| 1 | 0 | 1 | 0 | 1 | 0 | 0 | = | 84 |

Converting decimal to binary video:

HTTPS://WWW.YOUTUBE.COM/WATCH?HL=EN-GB&V=QWXIXU02ZQM&GL=SG

# Converting from base-10 to base-2 (Method 1)

Repeated Division: *Divide the decimal number by 2, writing down each remainder until a quotient of 0 is reached. The answer is read from the remainders, starting from the last to the first*

Eg: Determine the binary value of 55

55 / 2 = 27 remainder **1**

27 / 2 = 13 remainder **1**

13 / 2 = 6 remainder **1**

6 / 2 = 3 remainder **0**

3 / 2 = 1 remainder **1**

1 / 2 = 0 remainder **1**

Therefore the binary value of 55 = 1 1 0 1 1 1

# Converting from base-2 to base-10

| $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ | | |
|---|---|---|---|---|---|---|---|
| *32* | *16* | *8* | *4* | *2* | *1* | | |
| | | 1 | 0 | 0 | 1 | = | 9 |
| | | 1 | 0 | 1 | 1 | = | 11 |
| | 1 | 0 | 1 | 0 | 1 | = | 21 |
| 1 | 1 | 1 | 1 | 1 | 1 | = | 63 |

**Eg: $21 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$**

# Binary Arithmetic

- The basic rule of adding 2 binary digits

  0 + 0 = 0

  0 + 1 = 1

  1 + 0 = 1

  1 + 1 = 0 with a carry over of 1 to the next position to the left ( i.e. 1+1 = 10)

  1 + 1 + 1 = 1 with a carry over of 1 to the next position to the left (i.e. 1+1+1 = 11)

# Addition

Base-10

$$\begin{array}{cccc} 1 & 9 & 9 & 8 \\ + & & 1 & 1 \\ \hline 2 & 0 & 0 & 9 \end{array}$$

Base-2

$$\begin{array}{ccccc} & & 1 & 1 & \\ & 1 & 0 & 1 & 1 \\ + & & & 1 & 1 \\ \hline & 1 & 1 & 1 & 0 \end{array}$$

Note: A *binary digit* is called a ***bit***.
The bit with the **lowest exponent ($2^0$)** is called the ***least significant bit (LSB).***
The bit with the highest exponent is the ***most significant bit (MSB).***

# Binary Subtraction

- The basic rule of subtracting 2 binary digits

  0 - 0 = 0

  1 - 0 = 1

  1 - 1 = 0

  0 - 1 = 1

  (you must borrow “2” from the next significant bit from the left)

# Hexadecimal System (1)

- The binary number system is very important because of its connection with computers, but it is not the only other number system of importance.

- Hexadecimal (base-16) number systems provide a convenient shorthand representation for multi bit numbers in a digital system.

- A 4-bit string can be uniquely represented by one hex number

# Hexadecimal System (2)

- The hexadecimal number system is a base-16 number system and uses the digits 0 - 9 along with the letters A - F to represent numbers.

- It is used in:
  - Viewing the main memory and registers content
  - Expressing the colour in an HTML document, <body bgcolor = “#FFFFFF”> which is white

    <body bgcolor =“#000000”> which is black
  - Expressing address in a memory location eg 0000H , 0001H etc

- Convert hexadecimal (H) to decimal
  - $81F_{16}$ or 81FH (Hex 0-9,A-F)
- $= 8 \times 16^2 + 1 \times 16^1 + 15 \times 16^0$
- $= 2048 + 16 + 15$
- $= 2079_{10}$

- Exercise : Convert $259_{10}$ to hexadecimal (H)

| | | | |
|---|---|---|---|
| 16 | 259 | | |
| 16 | 16 | 3 | ← LSB |
| 16 | 1 | 0 | |
| | 0 | 1 | ← MSB |

Therefore,
$259_{10} = 103_{16}$

# Examples

| | | | |
|---|---|---|---|
| 16 | 257 | | |
| 16 | 16 | 1 ← | LSB |
| 16 | 1 | 0 | |
| | 0 | 1 ← | MSB |

| Binary (Base 2) | Decimal (Base 10) | Hexadecimal (Base 16) |
|---|---|---|
| $0000_2$ | $0_{10}$ | $0_{16}$ |
| $0001_2$ | $1_{10}$ | $1_{16}$ |
| $0010_2$ | $2_{10}$ | $2_{16}$ |
| $0011_2$ | $3_{10}$ | $3_{16}$ |
| $0100_2$ | $4_{10}$ | $4_{16}$ |
| $0101_2$ | $5_{10}$ | $5_{16}$ |
| $0110_2$ | $6_{10}$ | $6_{16}$ |
| $0111_2$ | $7_{10}$ | $7_{16}$ |
| $1000_2$ | $8_{10}$ | $8_{16}$ |
| $1001_2$ | $9_{10}$ | $9_{16}$ |
| $1010_2$ | $10_{10}$ | $A_{16}$ |
| $1011_2$ | $11_{10}$ | $B_{16}$ |
| $1100_2$ | $12_{10}$ | $C_{16}$ |
| $1101_2$ | $13_{10}$ | $D_{16}$ |
| $1110_2$ | $14_{10}$ | $E_{16}$ |
| $1111_2$ | $15_{10}$ | $F_{16}$ |
| | $16_{10}$ | |
| | $17_{10}$ | |
| | $18_{10}$ | |

# EXERCISES :

1. Convert $ABCD_{16}$ into binary.
   **$ABCD_{16}$ = 1010 1011 1100 $1101_2$** **(each hex digit is equal to 4 bits in binary)**

2. If a byte addressable RAM occupies the hexadecimal addresses $0000_{16}$ to $FFFF_{16}$.
   **How many address lines are needed**?

$\underbrace{0000}_{}{}_{16}$ to $\underbrace{FFFF}_{}{}_{16}$

0000 0000 0000 0000 to 1111 1111 1111 1111

**$A_0$**

**$A_{15}$ ……. $A_1$**

**16 address lines are needed**

| $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ | first eight binary powers |
|---|---|---|---|---|---|---|---|---|
| **128** | **64** | **32** | **16** | **8** | **4** | **2** | **1** | |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | =67 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | =255: the largest number represented in just one byte |
| 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | =12 |

**Advantages of using binary numbers**

- They have only two states 1 or 0 (on or off)
- Simple Arithmetic has only two states,
  i.e. 0+0, 0+1, 1+0, 1+1

# Example : Converting 00111101 base 2 to decimal

The number represented by $00111101_2$ is equal to $61_{10}$.
The value 61 is obtained using the method illustrated below:

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 1 |
| = | $(0 \times 128)$ + | $(0 \times 64)$ + | $(1 \times 32)$ + | $(1 \times 16)$ + | $(1 \times 8)$ + | $(1 \times 4)$ + | $(0 \times 2)$ + | $(1 \times 1)$ |
| = | | | 32 + | 16 + | 8 + | 4 + | | 1 |
| = | $61_{10}$ | | | | | | | |

# ENDIANNESS in FORENSICS

# Why is endianness important for forensics

- Commercial and open-source forensic tools interpret data from collected evidence and present in a human readable manner

- Forensic investigator need to dig deeper into hexadecimal and interpret it. Thus, performing surgery with raw bytes

# Understanding big-endian

- On processor that handle data in big-endian, the bytes are read from left to right and stored in memory starting at low address. The same as reading in English

| POSITION | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| BYTE | B1 | 39 | F5 | 46 |

This is how data would appear on memory where data is handled in big-endian.

B139F546……

# Understanding little-endian

- On processes that handle data in little-endian to store B1 39 F5 46, the bytes are read from right to left and stored in memory starting at the low address. The same as reading Arabic

| POSITION | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| BYTE | 46 | F5 | 39 | B1 |

This is how data would appear on memory where data is handled in little-endian.

46F539B1……

# References

https://en.wikibooks.org/wiki/A-level_Computing/AQA/Problem_Solving,_Programming,_Data_Representation_and_Practical_Exercise/Fundamentals_of_Data_Representation/Binary_number_system

http://www.ntu.edu.sg/home/ehchua/programming/java/DataRepresentation.html

http://www.bing.com/videos/search?q=number+conversion&&view=detail&mid=513C83B9264F09E49406513C83B9264F09E49406&FORM=VRDGAR